Microcontroller and Computer Architecture Project Proposal

Air Quality Monitoring System using ESP32
and MQ-135 Gas Sensor with a Web Dashboard

Submitted by:
Esha Rameel Rahman(MGP24EVLSI013)
E-mail: esharr.ev2428@saintgits.org Roll no: 13, S4
S Ardra(MGP24EVLSI023)
E-mail: ardra.ev2428@saintgits.org Roll no: 22, S4
Electronics Engineering(VLSI Design and Technology)

Department of Electronics Engineering
Saintgits College of Engineering, Kottayam

1 Introduction

Air pollution has become one of the most pressing environmental challenges of today. With rising industrialization, factories, and vehicle use, air pollution continues to be a leading cause of respiratory illness and damage. This project proposes the implementation of a low cost IoT-based Air Quality Monitoring System using an ESP32 Microcontroller integrated with an MQ-135 Gas Sensor. The hardware layer comprises MQ-135 gas sensor for air quality detection that provides both analog and digital signals representing the environmental condition.The ESP32 microcontroller serves as the central processing unit, reading sensor data through analog-to-digital converters (ADC) and digital pins. It performs data preprocessing, AQI calculation, threshold checking, and formats data for transmission. It connects to the internet via Wi-Fi and communicates with the cloud based server to transmit this data and display it in real time on the dashboard.

2 Block Diagram with Functional Description

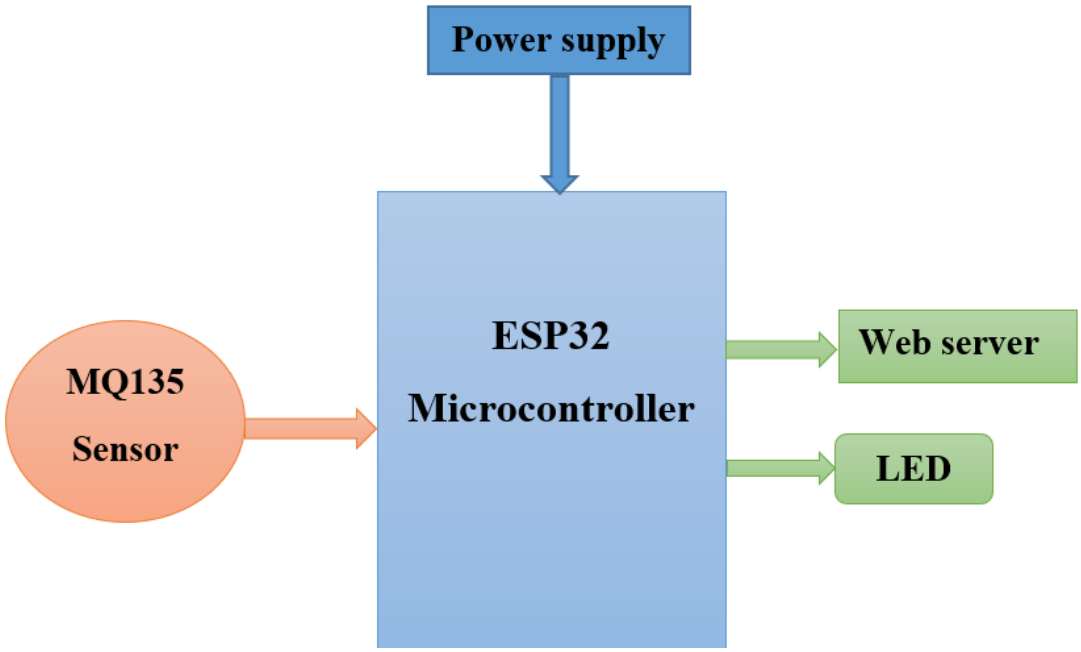


Figure 1: System Block Diagram of the Air Quality Monitoring System.

2.1 Functional Description of Each Block

MQ-135 Gas Sensor: This is the sensing layer. The MQ-135 is a semiconductor-based gas sensor capable of detecting harmful gases including NH_3, NO_x, CO_2, benzene, and smoke. It produces an analog voltage output proportional to gas concentration.

ESP-32 Microcontroller: The ESP32 serves as the system's central processor that does ADC sampling, PPM Conversion, Threshold evaluation, output control and wifi-transmission.

Green LED (GPIO27): Illuminates when air quality is acceptable (PPM < 190), indicating safe conditions. Turned off during warning or alert states.

Red LED (GPIO26): Illuminates during the toxic alert band ($\geq$191 PPM), providing a visible pollution indicator at all elevated pollution levels.

Buzzer (GPIO25): Activates with a 1-second pulse only when PPM reaches the toxic threshold ($\geq$191 PPM), providing an audible emergency alert in addition to the visual indicator.

Application Layer: The cloud-hosted backend receives the data from the esp and A web dashboard renders live AQI gauges, trend charts, and alert banners accessible from any browser.

3 Circuit Diagram with Component Details

3.1 Circuit Overview

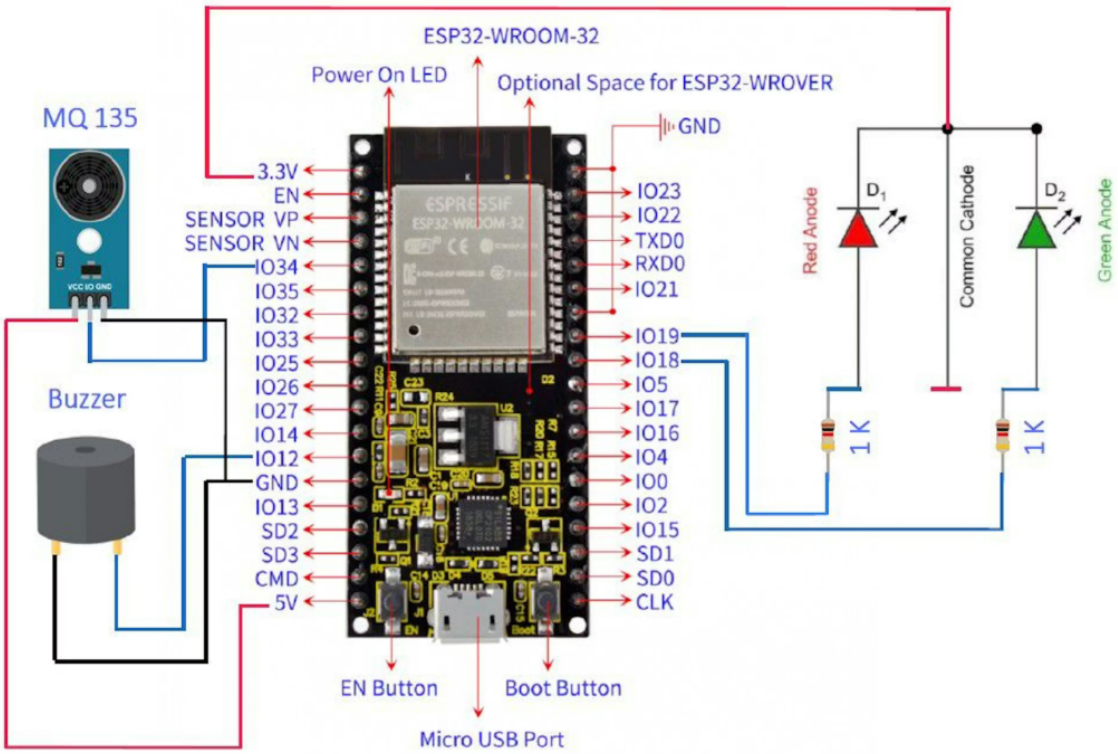


Figure 2: Circuit Block Diagram of the Air Quality Monitoring System.

3.2 Component Specifications

The system uses two primary components. The **ESP32** is a dual-core 32-bit microcontroller running at up to 240 MHz with 520 KB SRAM, 4 MB flash, a 12-bit SAR ADC, and built-in 802.11 b/g/n Wi-Fi. It operates on 3.3 V logic supplied via USB. In the firmware, GPIO34 is declared `INPUT` for ADC sampling of the MQ-135 analog output, while GPIOs 27, 26, and 25 are declared `OUTPUT` for the buzzer and LEDs. Both LEDs are initialised `HIGH` at startup, corresponding to the off state.

The **MQ-135 gas sensor** detects gases including NH_3, NO_x, CO_2, benzene, and smoke, operating on a 5 V supply with a 31 Ω heater element. Its AOUT pin produces an analog voltage proportional to gas concentration, which the `MQ135.h` library converts to a PPM value via `gasSensor.getPPM()`. A preheat period of over 24 hours is recommended for stable baseline readings.

The three output peripherals are a piezoelectric **buzzer** on GPIO25 (1 second pulse at ≥191 PPM), a **red LED** on GPIO26 (active above 190 PPM), and a **green LED** on GPIO27 (active below 190 PPM). Each LED is paired with a 1000 Ω current-limiting resistor, keeping forward current to approximately 5 mA, within the ESP32's 12 mA per-pin limit.

3.3 Pin Connection Summary

Table 1: Complete Pin Connection Table

Component	Component Pin	ESP32 Pin	Signal / Direction
MQ-135	VCC	5V Rail	Power in
MQ-135	GND	GND Rail	Power in
MQ-135	AOUT	GPIO34	Analog in
Buzzer	+ (via GPIO)	GPIO25	Digital out
Buzzer	−	GND Rail	Power
Green LED	Anode (1000 Ω)	GPIO27	Digital out
Green LED	Cathode	GND Rail	Power
Red LED	Anode (1000 Ω)	GPIO26	Digital out
Red LED	Cathode	GND Rail	Power

3.3.1 Output Peripherals and Supporting Components

- **Buzzer (GPIO25):** A piezoelectric buzzer is driven directly from GPIO25. It is energised for 1 second (`delay(1000)`) only when PPM $\geq$ 191, providing an audible toxic air alert.
- **Red LED (GPIO26):** Indicates poor or toxic air quality (PPM $>$ 190). Set `LOW` to illuminate.
- **Green LED (GPIO27):** Indicates acceptable air quality (PPM $<$ 190). Set `LOW` to illuminate.
- **Current-Limiting Resistors (1000 Ω):** One per LED, limiting forward current to approximately $(3.3 - 2.0)\,\mathrm{V}/1000\,\Omega \approx 5.9\,\mathrm{mA}$, within the ESP32 GPIO safe output current.

4 Code

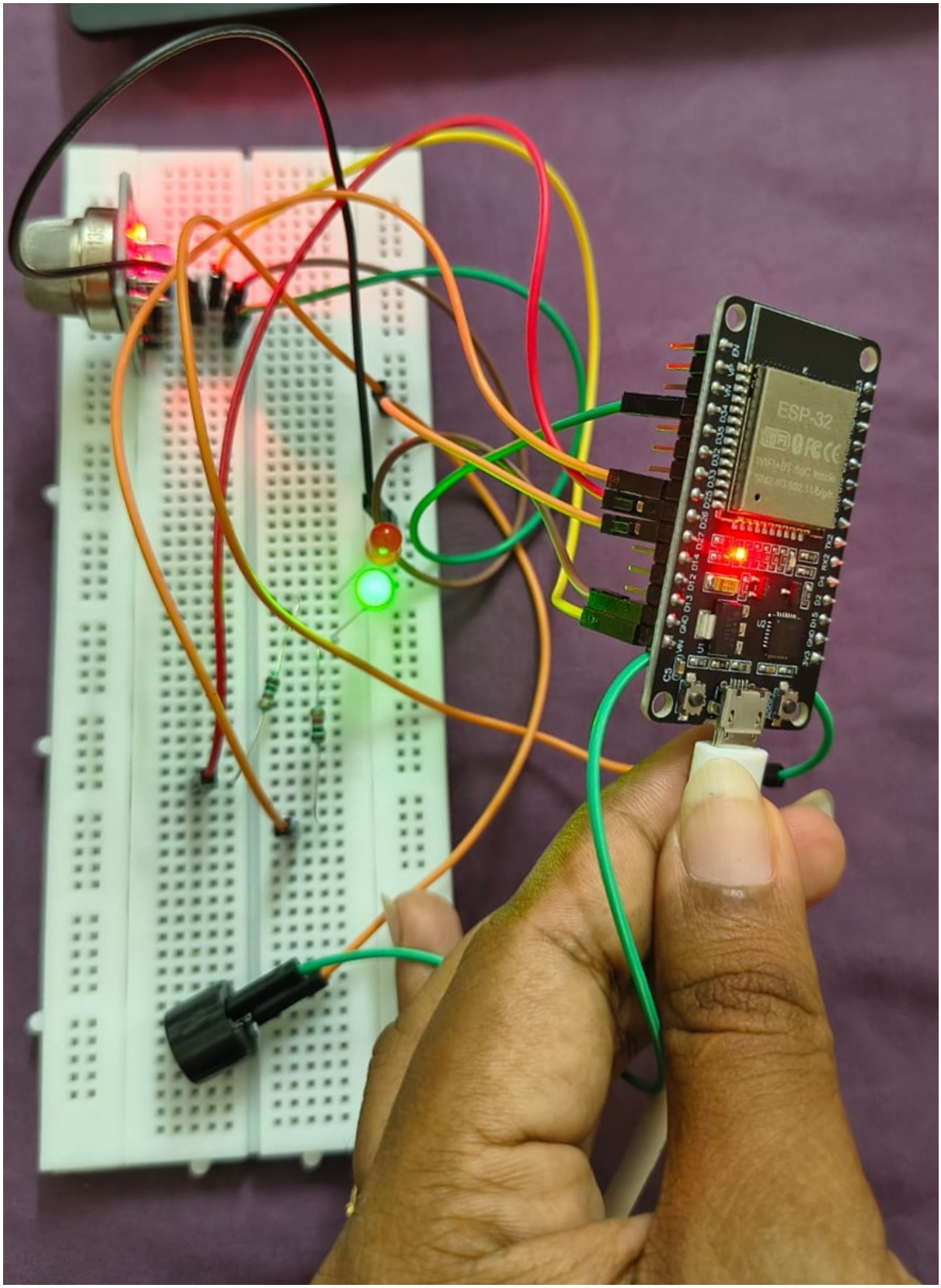
```
#define BLYNK_PRINT Serial
#define BLYNK_TEMPLATE_ID "TMPL3a3BaiCwV"
#define BLYNK_TEMPLATE_NAME "air quality monitoring"
#define BLYNK_AUTH_TOKEN "UdOVzjWkhpTCmR0_dkq7-VFpkuUFbbpd"
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <MQ135.h>


// --- User Configuration ---
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Esha";
char pass[] = "esha@123";

// --- Pin Definitions ---
#define MQ135_PIN       34   // Analog pin for MQ-135
#define BUZZER_PIN      27   // Digital pin for buzzer
#define RED_LED_PIN     26   // Digital pin for red LED (poor air)
#define GREEN_LED_PIN   25   // Digital pin for green LED (good air)
#define DHT_PIN         14   // Optional: DHT22 sensor pin

// --- Thresholds ---
#define POOR_AIR_THRESHOLD  190   // PPM of CO2 (calibrated) or raw ADC value

```

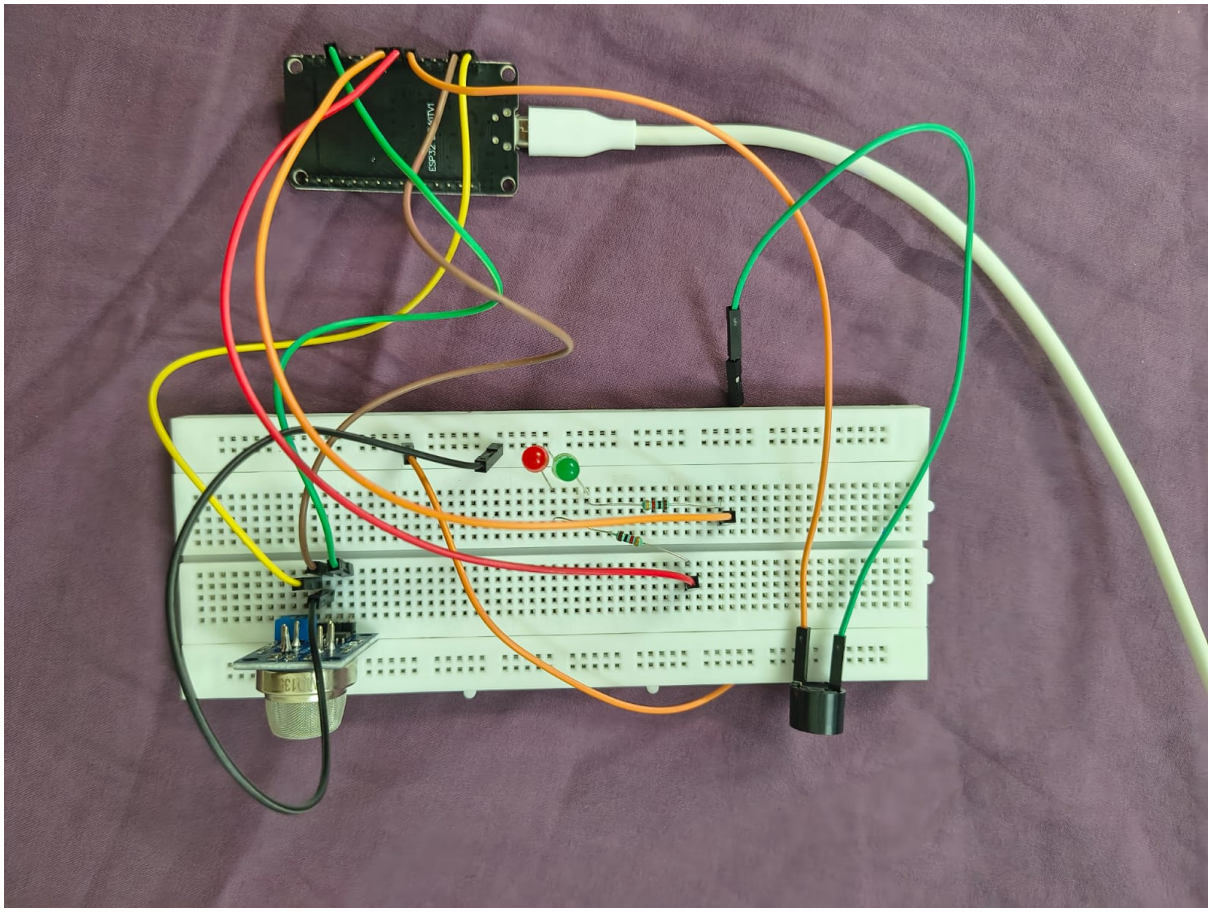
```

// --- Sensor Object ---
MQ135 mq135_sensor = MQ135(MQ135_PIN);
// DHT dht(DHT_PIN, DHT22); // Uncomment if using DHT

// --- Timing ---
unsigned long lastBlynkUpdate = 0;
const unsigned long updateInterval = 2000; // Send data every 2 seconds

void setup() {
  Serial.begin(115200);

  // Initialize pins
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);
  digitalWrite(RED_LED_PIN, LOW);
  digitalWrite(GREEN_LED_PIN, LOW);

  // Connect to Blynk
  Blynk.begin(auth, ssid, pass);
  Serial.println("Waiting for Blynk connection...");

```

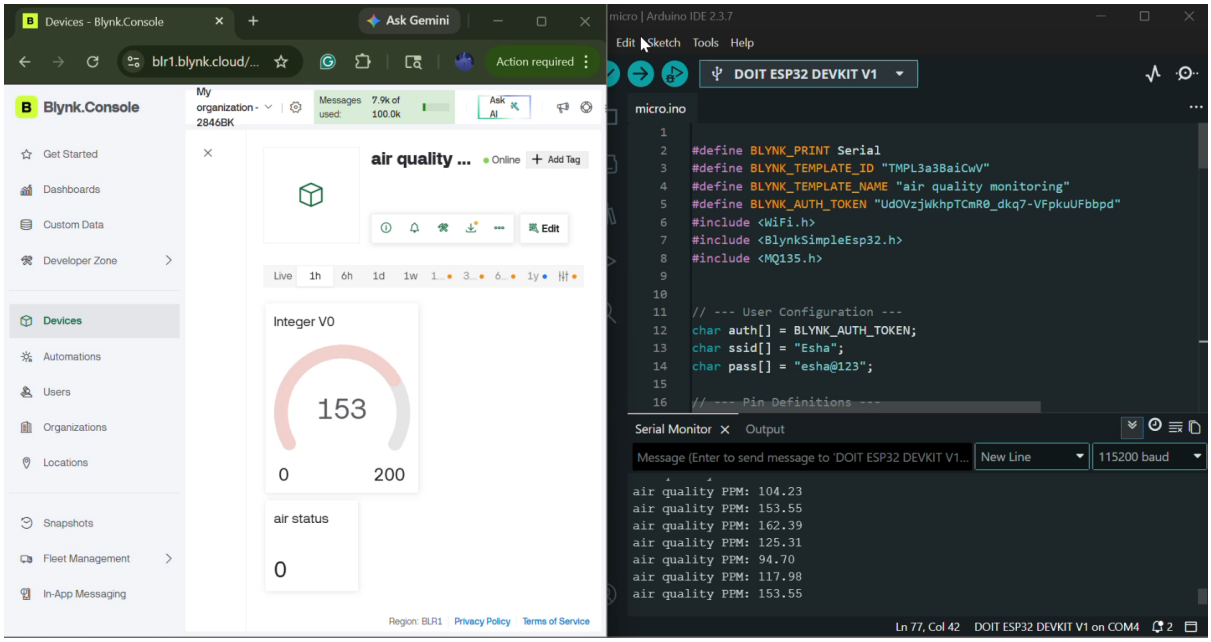
```
  // Optional: Initialize DHT sensor
  // dht.begin();

  // Let MQ-135 warm up (ideally 24h, but we give it 20s for demo)
  Serial.println("Sensor warming up...");
  delay(20000);
}

void loop() {
  Blynk.run(); // Keep Blynk connection alive

  if (millis() - lastBlynkUpdate > updateInterval) {
    lastBlynkUpdate = millis();

    // Read sensor
    float rzero = mq135_sensor.getRZero();
    float correctedPPM = mq135_sensor.getCorrectedPPM(25, 60); // Example: temp=25°C, humidity=60%

    // If using DHT, replace above with:
    // float temp = dht.readTemperature();
    // float hum = dht.readHumidity();
    // float correctedPPM = mq135_sensor.getCorrectedPPM(temp, hum);

    Serial.print("air quality PPM: ");
    Serial.println(correctedPPM);
```

```
    // Send data to Blynk Virtual Pins
    Blynk.virtualWrite(V0, correctedPPM);  // Send PPM to V0 (Gauge/Value Display)

    // Air quality assessment
    if (correctedPPM > POOR_AIR_THRESHOLD) {
      Blynk.virtualWrite(V1, 1);  // LED widget on V1 turns RED
      Blynk.virtualWrite(V2, 0);  // LED widget on V2 turns OFF
      digitalWrite(RED_LED_PIN, HIGH);
      digitalWrite(GREEN_LED_PIN, LOW);

      // Trigger buzzer for 1 second
      digitalWrite(BUZZER_PIN, HIGH);
      delay(1000);
      digitalWrite(BUZZER_PIN, LOW);

      Blynk.logEvent("air_quality_alert", "Poor Air Quality Detected!");
    } else {
      Blynk.virtualWrite(V1, 0);
      Blynk.virtualWrite(V2, 1);  // LED widget on V2 turns GREEN
      digitalWrite(RED_LED_PIN, LOW);
      digitalWrite(GREEN_LED_PIN, HIGH);
      digitalWrite(BUZZER_PIN, LOW);
    }
  }
}
```

5 Image of Working Model






Devices - Blynk.Console
Ask Gemini
blr1.blynk.cloud/...
Action required
Blynk.Console
My organization - 2846BK
Messages used: 7.9k of 100.0k
Ask AI
Get Started
Dashboards
Custom Data
Developer Zone
Devices
Automations
Users
Organizations
Locations
Snapshots
Fleet Management
In-App Messaging
air quality ...
Online
Add Tag
Edit
Live 1h 6h 1d 1w 1... 3... 6... 1y
Integer V0
153
0
200
air status
0
Region: BLR1
Privacy Policy
Terms of Service
micro | Arduino IDE 2.3.7
Edit Sketch Tools Help
DOIT ESP32 DEVKIT V1
micro.ino
#define BLYNK_PRINT Serial
#define BLYNK_TEMPLATE_ID "TMPL3a3BaiCwV"
#define BLYNK_TEMPLATE_NAME "air quality monitoring"
#define BLYNK_AUTH_TOKEN "UdOVzjWkhpTCmR0_dkq7-VFpkuUFbbpd"
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <MQ135.h>
// --- User Configuration ---
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Esha";
char pass[] = "esha@123";
// --- Pin Definitions ---
Serial Monitor
Output
Message (Enter to send message to 'DOIT ESP32 DEVKIT V1...
New Line
115200 baud
air quality PPM: 104.23
air quality PPM: 153.55
air quality PPM: 162.39
air quality PPM: 125.31
air quality PPM: 94.70
air quality PPM: 117.98
air quality PPM: 153.55
Ln 77, Col 42
DOIT ESP32 DEVKIT V1 on COM4

6 Expected Outputs and Outcomes

Table 2: Firmware Threshold Bands and System Outputs

Band	PPM Range	Green LED	Red LED	Buzzer
Fresh Air	< 190	ON (LOW)	OFF (HIGH)	OFF
Poor Air	>190	OFF (HIGH)	ON (LOW)	ON
Toxic Alert	≥ 190	OFF (HIGH)	ON (LOW)	1 s pulse

The system is expected to successfully detect and classify air quality into two bands — Fresh Air or Poor Air — in real time using the MQ-135 sensor, with the ESP32 responding instantly through the appropriate LED and buzzer outputs. The built-in web server should serve a live PPM reading and status page to any browser on the same Wi-Fi network, providing accessible, low-cost air quality monitoring without the need for any external cloud infrastructure. Overall, the project demonstrates how a minimal hardware setup costing under 1000 can deliver functional, real-time environmental monitoring with both physical and remote feedback mechanisms.